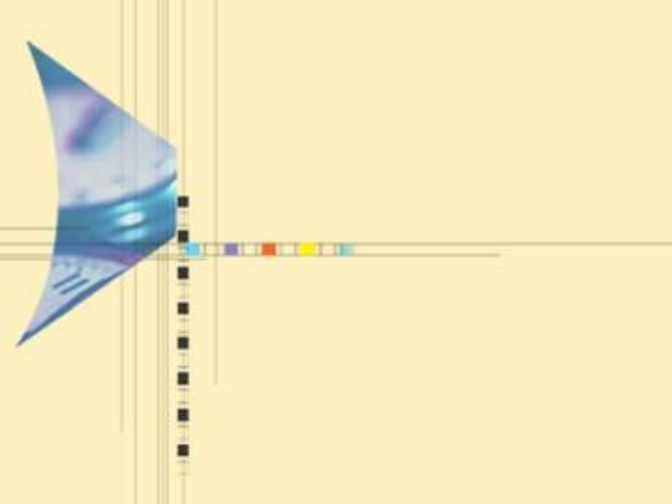




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

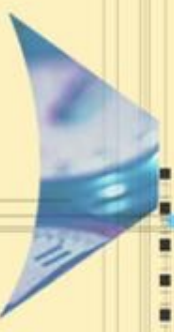
周一(3-4，三区1-502)，周三（6-7，三区1-325）

2025/4/30

武汉大学计算机学院

武汉大学
Wuhan University





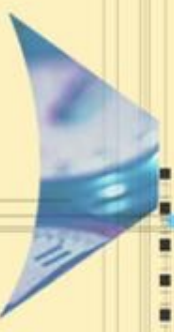
主要内容

10.1 引言

10.2 随机算法应用举例

10.3 随机游走算法





主要内容

10.2. 随机算法应用举例（选讲）

10.2.1 随机快速排序

10.2.2 随机选择

10.2.3 多选问题

10.2.4 随机采样

10.2.5 素数测试

10.2.6 测试串的相等性

10.2.7 模式匹配

10.2.8 货郎担问题随机算法



10.2.1 随机快速排序

- 随机快速排序或许是最流行的随机算法之一。
- 前面介绍过的快速排序算法 QUICKSORT, 假设输入元素的所有排列具有相等可能性, 该算法的平均运行时间为 $\Theta(n \log n)$; 如果输入已经有序或者几乎有序, 那么其运行时间为 $\Theta(n^2)$ 。
- 如何规避上述问题, 并确保算法期望运行时间为 $\Theta(n \log n)$?
 - (1) 引入预处理步骤 (时间为 $\Theta(n)$), 目的是随机排列这些元素。
 - (2) 直接在算法中引入随机性, 通过随机选择划分元素来进行数组的分割。随机选择划分元素就是为了放宽假设: 输入元素的所有排列具有相等可能性。

Algorithm 3.6 QUIUCKSORT

Input: An array $A[1..n]$ of n elements.

output: The elements in A sorted in nondecreasing order.

1. *quicksort*($A, 1, n$)

Procedure *quicksort*($A, low, high$)

1. **if** $low < high$ **then**

2. *SPLIT*($A[low..high], w$) { w is the new position
of $A[low]$ }

3. *quicksort*($A, low, w-1$)

4. *quicksort*($A, w+1, high$)

5. **end if**





RANDOMIZEDQUICKSORT

- 由于原来的QUICKSORT算法的数组划分算法SPLIT中，使用 $A[low]$ 作为划分元素。因此修改原算法，简单地在区间 $[low..high]$ 中随机选择一个下标 v ，并交换 $A[v]$ 和 $A[low]$ ，之后和原算法一样继续运行下去。
- 利用函数 $random(low, high)$ 返回一个介于 low 和 $high$ 之间的数。注意，任意介于 low 和 $high$ 之间的数以等概率 $1/(high-low+1)$ 产生。



Algorithm 10.3 RANDOMIZEDQUICKSORT

Input: An array $A[1..n]$ of n elements.

output: The elements in A sorted in nondecreasing order.

1. *rquicksort*($A, 1, n$)

Procedure *rquicksort*($A, low, high$)

1. **if** $low < high$ **then**

2. $v \leftarrow \text{random}(low, high)$

3. interchange $A[low]$ and $A[v]$

4. $SPLIT(A[low..high], w)$ { w is the new position
of $A[low]$ }

5. *rquicksort*($A, low, w-1$)

6. *rquicksort*($A, w+1, high$)

7. **end if**



随机快速排序算法分析

- 该算法在最坏情况的运行时间仍然是 $\Theta(n^2)$ ，但这个最坏情况不是由于输入形式，而是由于随机数生成器每次非常不幸地选择了不好的划分元素，这种情况不大可能发生。
- 无论输入的特性如何，该算法在 $O(n \log n)$ 时间内完成的概率都很高。事实上，输入的任何排列都不能引发其最坏的情况，算法的期望运行时间为 $\Theta(n \log n)$ 。

有效的拉斯维加斯算法

10.2.2 随机选择

- 在分治策略一讲中，已经证明算法3.4 SELECT的运行时间是 $\Theta(n)$ ，但是具有一个很大的常系数使得算法不切实际。尤其是对于小的和中等的 n 值，更是如此。
- 另外，SELECT算法通过两次取中间值，以帮助识别数组要丢弃的部分，导致算法流程复杂，理解起来困难。
- 修改SELECT算法，不再通过找到中值元素的中值来仔细选择要丢弃的部分，而是通过随机选择划分元素，不断随机丢弃输入数组的部分，直到剩下一个元素，即所要查找的第 k 小元素。这样得到一个简单，期望运行时间是一个带有小常系数的 $\Theta(n)$ 。

Algorithm 3.4 SELECT

Input: An array $A[1..n]$ of n elements and an integer $k, 1 \leq k \leq n$.

Output: The k th smallest element in A .

1. *select*($A, 1, n, k$)

Procedure *select*($A, low, high, k$)

1. $p \leftarrow high - low + 1$

2. **if** $p < 44$ **then** sort A and **return** ($A[k]$)

3. Let $q = \lfloor p/5 \rfloor$. Divide A into q groups of 5 elements each. If 5 does not divide p , then discard the remaining elements.

4. Sort each of the q groups individually and extract its median. Let the set of medians be M

5. $mm \leftarrow \text{select}(M, 1, q, \lfloor q/2 \rfloor)$ { mm is the median of medians}

6. Partition $A[low..high]$ into three arrays:

$A1 = \{A | A < mm\}$ $A2 = \{A | A = mm\}$ $A3 = \{A | A > mm\}$

7. **case**

$|A1| \geq k$: **return** *select*($A1, 1, |A1|, k$)

$|A1| + |A2| \geq k$: **return** mm

$|A1| + |A2| < k$: **return** *select*($A3, 1, |A3|, k - |A1| - |A2|$)

8. **end case**

算法 10.4 RANDOMIZEDSELECT

Input: An array $A[1..n]$ of n elements and an integer $k, 1 \leq k \leq n$.

Output: The k th smallest element in A .

1. $rselect(A, 1, n, k)$

Procedure $rselect(A, low, high, k)$

1. $v \leftarrow random(low, high)$

2. $x \leftarrow A[v]$

3. Partition $A[low..high]$ into three arrays:

$A1 = \{a | a < x\}$ $A2 = \{a | a = x\}$ $A3 = \{a | a > x\}$

4. **case**

$|A1| \geq k$: **return** $rselect(A1, 1, |A1|, k)$

$|A1| + |A2| \geq k$: **return** x

$|A1| + |A2| < k$: **return** $rselect(A3, 1, |A3|, k - |A1| - |A2|)$

5. **end case**

随机选择算法分析

- 在最坏情况下，随机数生成器在第j次迭代中选择第j小的元素，因此只丢弃一个元素。由于将A划分为A1、A2和A3所需的比较次数正好为n，因此算法在最坏情况下进行的元素比较总数为：

$$n + (n - 1) + \dots + 1 = \frac{n(n+1)}{2} = \Theta(n^2)$$

- 设C(n) 为对输入数组有n个元素时进行元素比较运算的期望次数。由于随机选择的v可以等概率假设为1,2, ..., n中的任何一个，因此根据v<k或v>k，有两种情况需要考虑。
 - 如果v<k：剩余元素的数量为n-v；
 - 如果v>k：剩余元素的数量为v-1
- 因此，元素比较的期望次数：C(n)<4n (P218页) 。
- 又由于每个元素至少被检查一次，因此C(n)≥n。
- 因此，RANDOMIZEDSELECT算法的期望运行时间为Θ(n)。

有效的拉斯维加斯算法

10.2.3 多选问题

- 设有 n 个元素的数组 $A[1..n]$ ，并且设 $K[1..r]$ ($1 \leq r \leq n$)是一个由1到 n 之间的 r 个正数构成的有序数组，即一个排位数组。
- 多选问题就是对于所有 i , $1 \leq i \leq r$ ，选择 A 中的第 $K[i]$ 小的元素。
- 假定 $A[1..n]$ 中所有元素各不相同。
- 如果 $r=1$ ，该问题即为选择问题。如果 $r=n$ ，该问题即为排列问题

多选问题确定算法基本思想

- 为方便描述, 设 $A[1..n]=[a_1, a_2, \dots, a_n]$; 有序序列 $K[1..r]=[k_1, k_2, \dots, k_r]$ 。
- 设中间排位为 $k=k_{[r/2]}$, 可使用SELECT算法发现第 k 小元素 a ;
- 接着, 根据 a 将 A 划分成两个子序列 A_1 和 A_2 , 其中元素分别对应小于 a 和大于 a 。令 $K_1=[k_1, k_2, \dots, k_{[r/2]}]$, $K_2=[k_{[r/2]+1-k}, k_{[r/2]+2-k}, \dots, k_r-k]$;
- 进行两次递归调用, 一个关于 A_1 和 K_1 , 另一个关于 A_2 和 K_2 。

算法 MULTISELECT

输入: n 元序列 $A=[a_1, a_2, \dots, a_n]$, 具有 r 个排位的有序序列 $K=[k_1, k_2, \dots, k_r]$.

输出: A 中第 k_i 小的元素, $1 \leq i \leq r$.

1. $\text{multiselect}(A, K)$

过程: $\text{multiselect}(A, K)$

1. $r = |K|$

2. **if** $r > 0$ **then**

3. 设置 $k = k_{\lfloor r/2 \rfloor}$

4. 用算法 SELECT 发现 A 中第 k 小元素 a

5. 输出 a

6. 令 $A_1 = [a_i \mid a_i < a]$, $A_2 = [a_i \mid a_i > a]$

7. 令 $K_1 = [k_1, k_2, \dots, k_{\lfloor r/2 \rfloor}]$, $K_2 = [k_{\lfloor r/2 \rfloor + 1} - k, k_{\lfloor r/2 \rfloor + 2} - k, \dots, k_r - k]$

8. $\text{multiselect}(A_1, K_1)$

9. $\text{multiselect}(A_2, K_2)$

10. **end if**

该算法的时间复杂度为 $\Theta(n \log r)$

多选问题随机算法基本思想

- 稍微修改随机快速排序算法，得到求解多选问题的随机算法。
- 将多选问题要寻找的元素成为靶标。
- 随机一致地挑选一个元素 a （而不是通过SELECT精心选择），将数组 A 关于 a 分割成两部分：一部分元素小于 a ，另一部分中元素大于 a ；
- 如果两部分均含有靶标，则递归调用算法继续求解。否则，丢弃不含有靶标的部分，对含有靶标部分继续求解。



算法10.5 QUICKMULTISELECT

输入: n 元序列 $A=[a_1, a_2, \dots, a_n]$, 具有 r 个排位的有序序列 $K=[k_1, k_2, \dots, k_r]$.

输出: A 中第 k_i 小的元素, $1 \leq i \leq r$.

1. $\text{qmultiselect}(A, K)$

过程: $\text{qmultiselect}(A, K)$

1. $r = |K|$

2. if $r > 0$ then

3. if $|A| = 1$ and $|K| = 1$ then 输出 a 并退出

4. 令 a 是随机一致选出的 A 中的一个元素

5. 将 A 中元素同 a 进行比较, 将 A 分成两个子序列 A_1 和 A_2 ,
 $A_1 = [a_i \mid a_i \leq a]$, $A_2 = [a_i \mid a_i > a]$ 。同时计算 $r(a)$, 即 a 在 A 中排位。

6. 将 K 分割为 $K_1 = [k \mid k \leq r(a)]$, $K_2 = [k - r(a) \mid k > r(a)]$

7. $\text{qmultiselect}(A_1, K_1)$

8. $\text{qmultiselect}(A_2, K_2)$

9. end if

该算法以概率 $1 - O(1/n)$ 的时间复杂度为 $O(n \log r)$

10.2.4 随机取样

- 问题描述:
 - 从一组 n 个元素中随机选择 m 个元素的样本，其中 $m < n$ 。为了简单起见，假设元素是1到 n 之间的正整数。
- 基本思想:
 - 首先将所有 n 个元素标记为未选中。
 - 接下来，重复以下步骤，直到选择了 m 个元素。
 - 生成一个介于1和 n 之间的随机数 r 。
 - 如果 r 被标记为未选中，则将其标记为选中并将其添加到样本中。

算法 10.6 RANDOMSAMPING

Input: Two positive integers m, n with $m < n$.

Output: An array $A[1..m]$ of m distinct positive integers selected randomly from the set $\{1, 2, \dots, n\}$.

```
1. Comment:  $S[1..n]$  is a boolean array indicating whether an
   integer has been selected.
2. for  $i \leftarrow 1$  to  $n$ 
3.      $S[i] \leftarrow \text{false}$ 
4. end for
5.  $k \leftarrow 0$ 
6. while  $k < m$ 
7.      $r \leftarrow \text{random}(1, n)$ 
8.     if not  $S[r]$  then
9.          $k \leftarrow k + 1$ 
10.         $A[k] \leftarrow r$ 
11.         $S[r] \leftarrow \text{true}$ 
12.    end if
13. end while
```

算法的期望运行时间 $T(n) = \Theta(n)$

RANDOMSAMPLING的使用方法

- 显然， m 和 n 之间的差值越小，运行时间就越长。
- 例如，如果 $n=1000$ 且 $m=990$ ，则算法将花费太多时间来生成样本中的最后整数，例如第990个整数。为了避免这个问题，我们可以随机选择10个整数，丢弃它们，并保留剩余的990个整数作为所需的样本。
- 因此，当具体使用该算法时，将假设 $m \leq n/2$ 。否则，可随机选择 $n-m$ 个整数，丢弃它们，并保留剩余的整数作为样本。

10.2.5 素数测试

- 问题描述: 给定一个正整数 n , 判断 n 是否为素数。
- 简单的方法: 给定一个输入数 n , 检查2到 $n-1$ 中的任何一个整数 m 是否被 n 整除。如果 n 能被任何一个 m 整除, 那么 n 就是合数, 否则它就是素数。
- 改进方法一: 若 n 是合数, 那么它可以被分解为两个数的乘积, 其中至少有一个必须小于或等于 $\sqrt{n}$ 。则一个有效的方法是测试 n 能否被2到 $\sqrt{n}$ 的数整除, 如果存在整除 n 的数, 那么 n 就是合数, 否则它就是素数。(见下页伪码)

反复相除找因子的测试方法

算法 10.7 BRUTE-FORCE PRIMALITYTEST

Input: A positive integer $n \geq 2$.

Output: true if n is prime and false otherwise.

1. $s \leftarrow \lfloor \sqrt{n} \rfloor$
2. **for** $i \leftarrow 2$ to s
3. **if** i divides n **then return** false
4. **end for**
5. **return** true

反复相除找因子的测试方法

- **改进方法二:** 所有的素数都是 $6k \pm 1$ 的形式，只有2和3是例外。则一个更有效的方法是先测试 n 是否能被2或3整除；然后测试 n 能否被所有形式为 $6k \pm 1 \leq \sqrt{n}$ 的数字整除。如果存在整除 n 的数，那么 n 就是合数，否则它就是素数。这样做的速度是测试2到 $n-1$ 所有数的3倍。

方法二有效性: 因为所有的整数都可以表示为 $6k+i$ ，其中 k 为任意整数， $i = -1, 0, 1, 2, 3, 4$ ；其中 $(6k+0)$ ， $(6k+2)$ ， $(6k+4)$ 可以被2整除； $(6k+3)$ 可以被3整除，只剩下 $6k \pm 1$ 。所以判断 n 是否是素数，要单独判断2、3，再判断 $6k \pm 1 \leq \sqrt{n}$ 的数。

反复相除找因子的测试方法

- 尽管改进方法相较于简单方法降低了计算复杂度，但算法的时间复杂度仍然是指数级的！
- 上述反复地除以数字的测试方法效率极低，唯一的优点是如果 n 是合数，则可以输出 n 的因子。事实上，对一个整数进行因式分解比仅仅测试它是素数还是合数要困难得多。

寻找“证据”的测试

- 在素数测试问题中，既然我们只需要做素数测试，我们采用寻找“证据”的思想，也就是证明一个数是合数。显然，找到 n 的一个因子就是它是合数的证据。但这样的证据非常少。
- 事实上，如果我们采取一个相当大的数字 n ，它的素因子的个数比从1到 n 之间的整数个数要少得多。
 - 众所周知，如果 $n < 2^k$ ，除非 k 非常小，除以 n 的不同素数的个数小于 $\pi(k) \approx k / \ln k$
- 因此，需要探求另外的“证据”。。。。

素数测试的随机算法

- 最流行的素数测试是随机测试。除了被测试的数 n 之外，这些测试还使用从某个样本空间随机选择的其他一些数 a ；
- 通常的随机素数检验从不将素数报告为合数，但合数有可能被报告为素数。
- 通过用几个独立选择的 a 值重复测试，可以降低错误概率；

素数测试的随机算法

- 素数的随机测试流程：
 1. 随机抽取一个数 a ;
 2. 检查涉及 a 和给定数 n 的一个等式（对应于所选测试）。如果等式不成立，则 n 是一个合数， a 被称为合数的“证据”（见证者），测试停止;
 3. 从第1步开始重复，直到达到所需的置信度。
- 经过多次迭代，如果没有发现 n 是一个合数，那么就可以认为它是一个素数。
- 怎么样的等式呢?

求幂运算

在讨论另一证据之前，首先介绍后续用到的求幂算法：
求 a 的 m 次幂

算法 10.8 EXPMOD

Input: Positive integers a , m and n with $m \leq n$.

Output: $a^m \pmod n$.

1. Let the binary digits of m be $b_k, b_{k-1}, \dots, b_0$
2. $c \leftarrow 1$
2. **for** $j \leftarrow k$ **downto** 0
3. $c \leftarrow c^2 \pmod n$
3. **if** $b_j = 1$ **then** $c \leftarrow ac \pmod n$
4. **end for**
5. **return** c

实例

求 a^5

- $5=101$, $b_2=1, b_1=0, b_0=1$

Algorithm 10.7 EXPMOD

Input: Positive integers a , m and n with $m \leq n$.

Output: $a^m \pmod{n}$.

1. Let the binary digits of m be $b_k, b_{k-1}, \dots, b_0$
2. $c \leftarrow 1$
2. **for** $j \leftarrow k$ **downto** 0
3. $c \leftarrow c^2 \pmod{n}$
3. **if** $b_j=1$ **then** $c \leftarrow ac \pmod{n}$
4. **end for**
5. **return** c

$$c=1$$

$$j=2$$

$$j=1$$

$$j=0$$

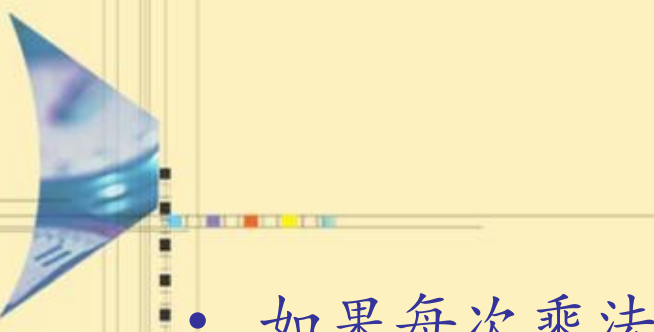
$$c=1$$

$$c=a^2$$

$$c=a^4$$

$$c=a$$

$$c=a^5$$

- 
- 如果每次乘法花费1个时间单元，算法EXPMOD的运行时间为 $\Theta(\log m) = O(\log n)$.
 - 然而，由于这里处理任意大的整数，因此要计算算法中执行的比特乘法的次数。两数相乘耗费 $O(\log^2 n)$ ，因此EXPMOD的运行时间是 $O(\log^3 n)$ 。



素数测试的随机算法

- 费马定理：如果 n 是素数，那么对于所有的 $a \not\equiv 0 \pmod{n}$ ，都有 $a^{n-1} \equiv 1 \pmod{n}$ 。
- 素数测试：检测 $2^{n-1} \equiv 1 \pmod{n}$ 是否成立。如是，输出“素数”；否则输出“合数”。

算法 10.8 PTEST1

Input: A positive odd integer $n \geq 5$

Output: prime if n is prime; otherwise composite

1. **if** $2^{n-1} \equiv 1 \pmod{n}$ then return prime {probably}
2. **else return** composite {definitely}

素数测试的随机算法

- 如果 n 为合数且算法输出“素数”，则称 n 为基2的伪素数。例如341。
- 虽然算法PTEST1只对 $a=2$ 进行测试，但令人惊讶的是算法PTEST1很少给出错误的结果。
- 例如：对于4到2000之间所有的合数，算法仅对341，561，645，1105，1387，1729和1905这样几个合数返回素数。此外，在小于100000的数中仅有78个数测试出错，他们中最大的是 $93961=7*31*433$ 。

PTEST1算法出错的情形

- 但是，的确对许多合数 n 的确存在一个整数 a 满足 $a^{n-1} \equiv 1 \pmod{n}$ 。换句话说，Fermat定理的逆是不正确的。
 - 一类被称为Carmichael数（卡迈克尔数）的合数 n ，它对于所有相对于 n 互素的正整数 a ，均满足Fermat定理。Carmichael数相当少，比如在小于 10^8 的范围内仅有255个。
- 当一个合数 n 对于底 a 满足Fermat定理时， n 被称为底 a 的伪素数。于是，算法PTEST1在 n 是素数或者是底2的伪素数时返回素数。

PEST2: PEST1算法的改进

- 改进算法PTEST1的一种方法是在2和n-2之间随机地选择底，这产生了算法PTEST2。

引理：如果（合数）n不是一个卡迈克尔数，那么算法PTEST2将检测出n为合数的概率至少为 $\frac{1}{2}$ 。

算法 10.9 PTEST2

Input: A positive odd integer $n \geq 5$

Output: prime if n is prime; otherwise composite

1. $a \leftarrow \text{random}(2, n-1)$

1. **if** $a^{n-1} \equiv 1 \pmod{n}$ then return prime {probably}

2. **else return** composite {definitely}



PTEST1/PTEST2算法的困难

- 遗憾的是，对卡迈克尔数，PTEST2仍然会错误地返回素数。最近已经证明了实际上有无穷多的卡迈克尔数。
- 如何设计一个更强大的随机化素性测试算法，绕过卡迈克尔数带来的困难？

素数的另一个必要条件

定理：如果 n 为奇素数，则方程 $x^2 \equiv 1 \pmod{n}$ 的根只有两个，即 $x = 1$ ， $x = -1$ （或 $x = n-1$ ）。

证明 $x^2 \pmod{n} \equiv 1$

$$\Leftrightarrow x^2 - 1 \equiv 0 \pmod{n}$$

$$\Leftrightarrow (x+1)(x-1) \equiv 0 \pmod{n}$$

- 由于 $n > 2$ ，所以由 $(x+1)(x-1) \equiv 0 \pmod{n}$ 可得 $x+1 \equiv 0 \pmod{n}$ 或 $x-1 \equiv 0 \pmod{n}$ ，但是不能同时成立。（如果同时成立，意味着 $((x+1)-(x-1)) \equiv 0 \pmod{n}$ 也成立，矛盾）
- 若 $x-1 \equiv 0 \pmod{n}$ 不成立，则 $\gcd(x-1, n)=1$ ，又 $x^2 \pmod{n} \equiv 1$ ，因此有 $x+1 \equiv 0 \pmod{n}$ ；类似的，若 $x+1 \equiv 0 \pmod{n}$ 不成立，则必有 $x-1 \equiv 0 \pmod{n}$ 。

因此，定理得证。即 $x = -1$ （ $n-1$ ）或 $x = 1$ 。

素数的另一个必要条件

方程 $x^2 \equiv 1 \pmod{n}$ 的根 $x = 1$ 或者 $x = -1$ (或 $x = n-1$) 称为平凡的; $x \neq \pm 1$ 的根为非平凡的.

推论: 如果方程 $x^2 \equiv 1 \pmod{n}$ 存在非平凡根, 则是合数.

合数的判别方法: 如果方程有非平凡的根, 则 n 为合数.

例如: $x^2 \pmod{12} \equiv 1 \Leftrightarrow x = 1$ 或 $x = 11$ 或 $x = 5$ 或 $x = 7$

结论: 由于 5 和 7 是非平凡的根, 12 是合数

测试方法

设 n 为奇素数, 则存在 q, m 使得 $n - 1 = 2^q m$, ($q \geq 1$).

构造序列:

$$a^m(\bmod n), a^{2m}(\bmod n), a^{4m}(\bmod n), \dots, a^{2^{q-1}m}(\bmod n)$$

其最后一项为 $a^{n-1}(\bmod n)$, 而且每一项是前面一项的平方.

测试方法如下:

- 对于任意 i ($i = 0, 1, \dots, q-1$), 判断 $a^{2^i m}(\bmod n)$ 是否为 1 和 $n-1$, 且它的后一项是否为 1.
- 如果其后项为 1, 但本项不等于 1 和 $n-1$, 则它就是非平凡的根, 从而知道 n 不是素数.
- 随机选择 $a \in \{2, 3, \dots, n-1\}$, 进行上述测试.

实例

例如： $n=561$, $n-1=560=2^4 \times 35$, 假设 $a=7$, 构造的序列为

$$7^{35} \pmod{561} = 241,$$

$$7^{2^{135}} \pmod{561} = 7^{70} \pmod{561} = 298,$$

$$7^{2^{235}} \pmod{561} = 7^{140} \pmod{561} = 166,$$

$$7^{2^{335}} \pmod{561} = 7^{280} \pmod{561} = 67,$$

$$7^{2^{435}} \pmod{561} = 7^{560} \pmod{561} = 1$$

第5项为1，但是第4项等于67，它既不等于1也不等于560，是个非平凡的根，因此可以判定 n 为合数。

根据这个思想设计的素数测试算法称为Miller-Rabin算法，它随机选择正整数 $a \in \{2, 3, \dots, n-1\}$ ，然后进行上述测试。

算法子过程(一)

算法10.10 FINDq-m(n) //找 q, m 使得 $n-1=2^q m$

1. $q \leftarrow 0; m \leftarrow n-1$
2. repeat
3. $m \leftarrow m/2$
4. $q \leftarrow q+1$
5. until m 是奇数

运行时间: $O(\log n)$



算法子过程（二）

算法10.11 PEST3(n, q, m) //检测序列是否存在非平凡的根

1. $a \leftarrow \text{Random}(2, n-1)$
2. $x_0 \leftarrow \text{EXPMOD}(a, m, n)$ // $x_0 = a^m \pmod n$, $O(\log^3 n)$
3. for $i \leftarrow 1$ to q do // $q = O(\log n)$
4. $x_i \leftarrow x_{i-1}^2 \pmod n$ // $O(\log^2 n)$
5. if $x_i = 1$ and $x_{i-1} \neq 1$ and $x_{i-1} \neq n-1$
6. then return composite {肯定}
7. if $x_q \neq 1$ then return composite {肯定}
8. return prime {可能}

性能分析： 1 次测试运行时间 $O(\log^3 n)$ ；可证明1次测试出错的概率至多 $1/2$ 。重复运行 k 次，可将出错概率降到至多 2^{-k} 。

PTEST3算法分析

定理：如果算法PTEST3返回composite，那么n是合数。

- 注意上述定理的逆否命题是：如果n是素数，那么算法PTEST3返回prime，这表示算法在n是素数的时候从不会出错。
- 显然，PTEST3在处理非卡迈克尔数是同PTEST2同样好。可以证明算法PTEST3在处理一个卡迈克尔数时出错的概率最大为1/2。因此，算法PTEST3对于任意合数出错的可能性最大是1/2。
- 这样，通过反复测试k次，出错的概率最多是 2^{-k} 。如果 $k = \lceil \log n \rceil$ ，失败的概率变成不超过 $2^{-\lceil \log n \rceil} \leq 1/n$ 。

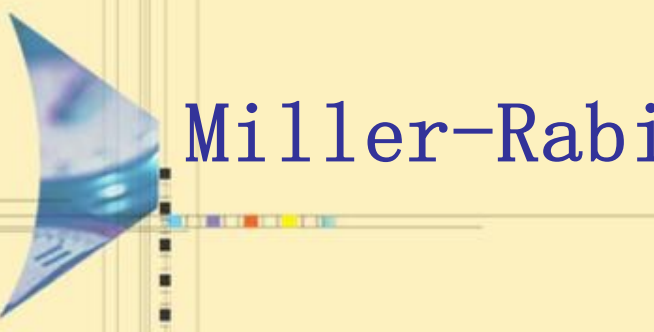
Miller-Rabin算法

令 $k = \lceil \log n \rceil$, 出错的概率小于等于 $2^{-k} \leq 1/n$. 即算法给出正确答案的概率至少为 $1 - 1/n$. 换句话说, 如果 n 为素数, 则算法输出素数; 如果 n 为合数, 则算法以至少 $1 - 1/n$ 的概率输出“合数”.

算法10.12 PRIMALITYTEST(n) // $n \geq 5$, 奇整数

1. FINDq-m(n)
2. $k \leftarrow \lceil \log n \rceil$
3. for $i \leftarrow 1$ to k //重复执行 $\log n$ 次
4. PTEST3(n, q, m)

时间: $T(n) = O(\log^4 n)$ // 按位乘统计



Miller-Rabin算法

定理：在处理一个奇整数 $n \geq 5$ 时，在时间 $O(\log^4 n)$ 内，算法PRIMALITYTEST运行如下：

- (1) 如果 n 是素数，那么它输出prime；
- (2) 如果 n 是合数，那么它至少以 $1 - \frac{1}{n}$ 的概率输出composite。

10.2.6 测试串的相等性

- 问题描述：
 - 假定A和B两个组能够通过一条信道来通信，并且假定信道非常可靠；
 - A有一个很长的串 x ，B有一个很长的串 y ；
 - 现在要确定是否 $x = y$ 。
- 显然，A可以把 x 发送给B，B可以接着立刻判断是否 $x = y$ 。
- 从信道的利用率来看，这种方法是十分浪费的。

随机测试方法

- 基本思想
 - A可以从x中取出一个短得多的串作为x的“指纹”，并且把指纹发送给B；
 - B用同样的方法获得y的“指纹”；
 - 如果这两个指纹相等，则B就假定 $x=y$ ，否则B就得出 $x \neq y$ 的结论。
- 优点：只需要在信道上传输一个很短的串。

指纹生成方法

- 对于一个比特串 w ，设 $I(w)$ 是比特串 w 表示的一个整数。
- 一种收集指纹的方法是选一个素数 p ，然后使用指纹函数：

$$I_p(x) = I(x) \pmod{p}$$

- 如果 p 不是太大的话，则指纹 $I_p(x)$ 可以作为一个短串发送，传送的比特数是 $O(\log p)$ 。

假匹配

- 如果 $I_p(x) \neq I_p(y)$ ，那么显然 $x \neq y$ 。但是逆否命题不成立。即如果 $I_p(x) = I_p(y)$ ，不一定有 $x = y$ 的结果。我们称这种现象为假匹配。
- 一般地，如果 $x \neq y$ ，但是 $I_p(x) = I_p(y)$ ，即 p 整除 $I(x) - I(y)$ 时就发生了假匹配。
- 对于固定的 p ，存在一定的串 x 和 y 偶对，对于它们这种指纹生成方法总是要失败的。

解决假匹配问题的方法

- 在每次检查两个串的相等性时随机地选择 p ，而不是预先选好，这样可以避开存在的 x 和 y 这些偶对的问题。
- 而且，随机选择 p 也允许重传另一个指纹，这样在 $x = y$ 的情况增加了可信度。

测试串的相等性的随机算法

算法 10.13 STRINGQUALITYTEST

1. A chooses p at random from the set of primes less than M .
2. A sends p and $I_p(x)$ to B.
3. B checks whether $I_p(x) = I_p(y)$ and confirms the equality or inequality of the two string x and y .



假匹配的概率分析

- 设 n 是串 x 的二进制表示形式的比特数，当然， n 也等于串 y 的二进制表示形式的比特数，否则问题就是平凡的。
- 设 $\pi(n)$ 是小于 n 的不同素数的个数，众所周知， $\pi(n)$ 趋近于 $n/\ln n$ 。(素数定理)
- 又知道如果 $k < 2^n$ ，则除非 n 非常小，否则能够整除 k 的不同素数的个数小于 $\pi(n)$ 。
- 由于失败仅在假匹配的情况下发生，即 $x \neq y$ 但 $I_p(x) = I_p(y)$ 时，而这只有在 p 整除 $I(x) - I(y)$ 时可能发生。

假匹配的概率分析

- 因此，对于两个n为串x和y，失败的概率是

$$\frac{|\{p \mid p \text{ is a prime } \leq M \text{ and } p \text{ divides } I(x) - I(y)\}|}{\pi(M)} \leq \frac{\pi(n)}{\pi(M)}$$

- 如果选择 $M = n^2$,则可以得到

$$\text{probability}(\text{failure}) \leq \frac{\pi(n)}{\pi(M)} \approx \frac{n / \ln(n)}{n^2 / \ln(n^2)} = \frac{\ln(n^2)}{n \ln n} = \frac{2}{n}$$

- 更进一步，如果重复执行算法k次，每次随机选择一个小于M的素数，则概率变成最大是 $(2/n)^k$ 。

举例

- 假定 x 和 y 都是100万比特串，即 $n=1\,000\,000$ ，则 $M=10^{12} = 2^{39.8631}$ 。
- 在这种情况下，需要传输 p 的比特数最多是 $\lfloor \log M \rfloor + 1 = 39 + 1 = 40$ ，需要传输的 x 的指纹位数最多是 $\lfloor \log(p-1) \rfloor + 1 \leq \lfloor \log M \rfloor + 1 = 40$ 。这样，传输比特总数最多是80。
- 一次传输失败的概率最多是 $2/n = 2/1\,000\,000$ 。因为 $\lfloor \log \log n \rfloor = 5$ ，重复算法5次将假匹配的概率减小到几乎等于 $n^{-\lfloor \log \log n \rfloor} = (10^6)^{-5} = 10^{-30}$ ，是很少的了。

10.2.7 模式匹配

问题描述:

- 给出一个文本串 $X = x_1x_2 \dots x_n$ 和模式 $Y = y_1y_2 \dots y_m$, 其中 $m \leq n$ 。不失一般性, 假定文本字母表是 $\Sigma = \{0, 1\}$ 。
- 确定模式 Y 是否出现在文本串 X 中。

直接方法:

- 简单地移动模式经过整个文本, 在每个位置将长度为 m 的部分文本串和模式作比较。
- 最坏情况下, 每次移动比较 m 次, 需移动 $(n-m+1)$ 次, 运行时间是 $O(nm)$ 。

随机算法的模式匹配

基本思想:

- 与直接方法一样，把模式Y划过整个文本串X;
- 但同直接方法比较模式Y与 $X(j) = x_j x_{j+1} \dots x_{j+m-1}$ 不同，随机算法比较模式指纹 $I_p(Y)$ 和文本块指纹 $I_p(X(j))$ 。

经过推导可知，文本块的指纹易于计算，新块 $X(j+1)$ 的指纹可以从 $X(j)$ 的指纹计算出来。

指纹的计算

$$I_p(X(j)) \\ = (2^{m-1}x_j + 2^{m-2}x_{j+1} + \cdots + 2^{m-m}x_{j+m-1})(\text{mod } p)$$

$$I_p(X(j+1)) \\ = (2^{m-1}x_{j+1} + 2^{m-2}x_{j+2} + \cdots + 2^{m-m}x_{j+m})(\text{mod } p)$$

$$\text{得: } I_p(X(j+1)) = (2I_p(X(j)) - 2^m x_j + x_{j+m})(\text{mod } p)$$

$$\text{设 } W_p = 2^m(\text{mod } p)$$

$$\text{得: } I_p(X(j+1)) = (2I_p(X(j)) - W_p x_j + x_{j+m})(\text{mod } p)$$

模式匹配的随机算法

算法 10.14 : PATTERNMATCHING

输入: 长度为 n 的文本串 X 和长度为 m 的模式 Y .

输出: 如果 Y 在 X 中, 则返回 Y 在 X 中的第一个位置; 否则返回0.

1. 从小于 M 的素数集中随机选择素数 p
2. $j \leftarrow 1$
3. 计算 $W_p = 2^m \pmod p$, $I_p(Y)$, $I_p(X_j)$
4. while $j \leq n - m + 1$
5. if $I_p(X_j) = I_p(Y)$ then return j // (可能)找到了匹配
6. $I_p(X(j+1)) \leftarrow (2I_p(X(j)) - W_p x_j + x_{j+m}) \pmod p$
7. $j \leftarrow j + 1$
8. end while
9. return 0 // Y (肯定)不在 X 中

PATTERNMATCHING算法分析

- 每个 W_p , $I_p(Y)$, $I_p(X(1))$ 的计算, 耗费 $O(m)$ 的时间;
- 对于 $2 \leq j \leq n - m + 1$, 每个 $I_p(X(j))$ 的计算仅需 $O(1)$ 的时间, 总共需要 $O(n)$ 的时间;
- 整体运行时间是 $O(n+m)$ 。

假匹配概率分析

- 假匹配：仅当对于某个 j ，
有 $Y \neq X(j)$ 但 $I_p(Y) = I_p(X(j))$ ；

- 假匹配发生情况：如果被选的素数 p 整除

$$\prod_{\{j|Y \neq X(j)\}} |I(Y) - I(X(j))|$$

- 这个乘积不可能超过 2^{mn} ，因此整除它的素数个数不可能超过 $\pi(mn)$ 。令 $M = 2mn^2$ ，假匹配概率不可能超过

$$\frac{\pi(mn)}{\pi(M)} \approx \frac{mn / \ln(mn)}{2mn^2 / \ln(2mn^2)} = \frac{\ln(2mn^2)}{2n \ln(mn)} < \frac{\ln(mn)^2}{2n \ln(mn)} = \frac{1}{n}$$

- 假匹配概率只和文本串长度 n 有关，而模式长度无关。

10.2.8 货郎担问题

问题描述

- 假设有一个旅行商人要拜访 n 个城市，选择路径的限制是每个城市只能拜访一次，而且最后要回到原来出发的城市。
- 路径的选择目标是 路径总路程为所有路径之中的最小值。

传统方法

贪心算法、蛮力算法、动态规划法、回溯法、分支定界法

货郎担问题的随机算法

- 设已知城市数 n 、每两城之间距离，若两城不直接相通，两者距离为 ∞ ；
- 生成符合限制的随机路径，计算路径总路程，更新目前最优的随机路径；
- 经过大量计算，很有可能得到最优的随机路径。

注意：

- 解决TSP问题的随机算法属于蒙特卡罗算法，即一定有解，但有可能是错解。
- 因为由于路径是随机的，也有可能每次随机生成都错过了最优解。

算法 10.15 - RandomTSP

输入: 城市数 n 、每两城市间距离.

输出: 解决TSP问题的最优路径和路径总长

1. $bestPath \leftarrow []$, $bestDistance \leftarrow \infty$, $iteration \leftarrow 100000$
2. 初始化随机路径 $path \leftarrow [1, 2, \dots, n]$
3. $distance \leftarrow$ 计算随机路径 $path$ 的总长度
4. **while** $j < iteration$
5. **if** $distance < bestDistance$
6. **then** $bestPath \leftarrow path$ // 更新最优解
7. $bestDistance \leftarrow distance$
8. $path \leftarrow$ 生成新的随机路径
9. $distance \leftarrow$ 计算随机路径 $path$ 的总长度
10. $j \leftarrow j + 1$
11. **end while**
12. **return** $bestPath, bestDistance$ //(可能)找到了最优解